Arreglos de Numpy

Estadística con Python

Mario A. Morales Rivera
mamorales@correo.unicordoba.edu.co
Universidad de Córdoba



Tabla de contenidos I

1. Introducción
2. Creación
3. Matrices
4. Secuencias regulares
5. Tipos de datos
6. Indexación



Introducción



Arreglos

- Los arreglos en Python se pueden entender como una variante de las listas donde todos los elementos deben ser del mismo tipo.
- En el paquete básico (o estándar) de Python hay un tipo de objeto llamado `array`, pero este no es eficiente para cálculos estadísticos y matemáticos, por tanto no lo trataremos en este libro.
- Estudiaremos los arreglos definidos en el paquete *Numerical Python*, ampliamente conocido como **NumPy**.
- Este módulo debe estar instalado previamente y se carga con el comando `import numpy`.



Características de los arreglos

- Operaciones *vectorizadas*. No es necesario el uso de bucles para recorrer los elementos del arreglo.
- Los arreglos con un solo índice (o un índice unidimensional) se conocen como vectores.
- Arreglos con dos índices (o un índice bidimensional) se usan como una eficiente estructura de datos, matrices y tablas.
- Los arreglos pueden tener mas de dos índices, es decir, pueden ser n—dimensionales.



Creación



Creación de arreglos de numpy

- Primero cargamos el paquete `numpy`, lo haremos con el nombre `np` como es la costumbre en la comunidad de usuarios de python.

```
import numpy as np                                    ①
t=(2,3,4,1)
at=np.array(t)
l2=[7.5,8.1,9]
al=np.array(l2)
print(at,type(at))
```

① Cargamos el módulo `numpy`.

```
[2 3 4 1] <class 'numpy.ndarray'>
```



Creación de arreglos de `numpy` (II)

- En la salida anterior `ndarray` significa *arreglo* n dimensional.
- Con el siguiente código se crea un arreglo bidimensional, una matriz de 2 filas y 3 columnas, observe que se entrega una lista con dos listas anidadas de la misma longitud, las listas anidadas pasan a ser las filas de la matriz.
- Análogamente se pueden crear arreglos a partir de tuplas anidadas



Creación de arreglos de numpy (III)

```
a2=np.array([[1, -1  , 0], [0, 1, -1]])   ①
ta2=np.array(((2,3),(4,1)))               ②
```

① Matriz o arreglo de dimensión 2×3 creado a partir de listas anidadas.
② Matriz o arreglo de dimensión 2×2 creado a partir de tuplas anidadas.



Creación de arreglos de numpy (IV)

Con los métodos `.ndim` y `.shape` se obtiene el número de dimensiones y la longitud de cada dimensión de un arreglo de `numpy`

```
nd=a2.ndim                    ①
a2.shape                      ②
```

① Número de dimensiones del arreglo.
② Longitud de cada dimensión

```
(2, 3)
```



Creación de arreglos de `numpy` (V)

- Note que `a2.shape` regresa una tupla cuyas componentes son las longitudes de cada dimensión.
- De esta forma, para el caso de arreglos con dos dimensiones, es decir matrices, podemos obtener el número de filas mediante `a2.shape[0]`, y el número de columnas con `a2.shape[1]`.
- El método `.reshape` sirve para crear arreglos n—dimensionales a partir de un arreglos unidimensionales.



Matrices



Creación de matrices

Veamos un ejemplo, ingresar la matriz

$$A = \begin{bmatrix} 9 & 137 & 297 & 261 \\ 136 & 2114 & 4176 & 3583 \\ 269 & 4176 & 8257 & 7104 \\ 260 & 3583 & 7104 & 12276 \end{bmatrix} \tag{1}$$



Creación de matrices (II)

Como un arreglo bidimencional 4×4 de numpy

```
Aa=np.array([9,137,297,261,
             136,2114,4176,3583,
             269,4176,8257,7104,
             260,3583,7104,12276])                    ①
#
A=Aa.reshape(4,4)                                     ②
print(A)
```

① Los elementos de la matriz en un arreglo unidimensional de 16 filas y una columna.

② Se modifican las dimensiones, se transforma el arreglo en una matriz de 4×4.

```
[[   9   137   297   261]
 [ 136  2114  4176  3583]
 [ 269  4176  8257  7104]
```



Creación de matrices (III)

- Note que de forma predeterminada el llenado de la matriz se hace por filas
- En caso que se requiera, podemos conseguir que el llenado de la matriz se haga por columna, en ese caso se obtendría la transpuesta de la matriz 1 dada en el ejemplo.

```
np.reshape(Aa,(4,4),order="F")
```

```
array([[   9,   136,   269,   260],
       [ 137,  2114,  4176,  3583],
       [ 297,  4176,  8257,  7104],
       [ 261,  3583,  7104, 12276]])
```



Observación

Nota

no se confunda, la `F` **no** es de fila, es de `Fortran`, lenguaje de programación de alto nivel que se usa principalmente en aplicaciones científicas y matemáticas



Arreglos especiales

- El paquete `numpy` tiene métodos para creación de arreglos especiales, por ejemplo, arreglos de ceros, de unos, en general arreglos con algún valor repetido, con cierta repetición de series de números, con números de un tipo específico de datos (`dtype`).
- A continuación ilustramos el uso de las funciones `zeros`, `zeros_like`, `ones`, `ones_like`, `linespace` y `arrange` para crear arreglos con valores y tipos de datos específicos



Arreglos especiales (II)

```
a0s=np.zeros(5)                    ①
c=np.zeros_like(l2)                ②
u=np.ones(4)                       ③
t2=((2,3),(4,1))
u2=np.ones_like(t2)                ④
```

① Arreglo de 5 ceros.
② Arreglo de ceros con las dimensiones y tipo de datos de `l2`.
③ Arreglo de 4 unos.
④ Arreglo de unos con las dimensiones y tipo de datos de `t2`.



Secuencias regulares



La función `linespace()`

La función `linespace()` crea un arreglo con n elementos con valores separados uniformemente en el intervalo $[p, q]$, estos arreglos son muy útiles una aplicación es momento de graficar, para generar el conjunto de valores del dominio de la función

```
a=np.linspace(0,1,5)
print(a,type(a))
```

```
[0.   0.25 0.5  0.75 1.  ] <class 'numpy.ndarray'>
```

Observe que el arreglo anterior, creado por `linespace()`, tiene $n = 5$ elementos, inicia en $p = 0$ y termina en $q = 1$.



La función arrange()

arrange() se usa de manera similar a la función range() que estudiamos en la clase de listas. Se pueden generar secuencias de números reales (no solo enteros) con incrementos que también pueden ser números reales.

```
np.arange(1.2,6,0.5)
```

```
array([1.2, 1.7, 2.2, 2.7, 3.2, 3.7, 4.2, 4.7, 5.2, 5.7])
```

En el anterior ejemplo, 1.2 es el valor de corresponde al valor de inicio, start, en caso que se omita el arreglo iniciará en cero, 6 es el valor de final stop de la secuencia, este valor es excluido, es decir el arreglo generado no lo incluirá.



Matrices especiales

El paquete `numpy` cuenta con funciones para la creación de matrices especiales, como la identidad, nula, escalar entre otras. En cuanto a la matriz identidad, hay dos funciones para crearla: `eye()` e `identity()`

```
id=np.eye(2)                    ①
id2=np.identity(3)              ②
E2=2*id2                        ③
```

① Matriz idéntica de orden 2.
② Matriz idéntica de orden 3.
③ Matriz escalar $2 \times I_3$.



Matrices especiales (II)

Las matrices escalares son de la forma $c \times I$, donde I es la identidad, podemos crear matrices escalares multiplicando la identidad por la constante c

La diferencia entre `eye()` e `identity()` es que con `eye()` se puede indicar, con el argumento `k`, cuál de las diagonales de la matriz se llenará de unos, de manera predeterminada `k=0`, con lo que se genera la identidad, pero si se indica `k=1`, se llena con unos la primera diagonal por encima de la diagonal principal, si `k<0` se llenan las diagonales por debajo de la principal.



Matrices especiales (III)

A continuación se crea un arreglo bidimensional de 4×4 con todos los elementos iguales, en este caso, a 1.

Estas matrices tienen la particularidad de que al multiplicarlas por otra, el efecto es sumar las filas (o columnas) de la segunda matriz, algunos autores les llaman matrices `sumadoras`

```
S4=np.full((1,4),1)    ①
S4@A                   ②
```

① Vector fila (1×4) de puros unos.
② Suma de las filas de la matriz A dada en 1.

```
array([[  674, 10010, 19834, 23224]])
```



Tipos de datos



Tipos de datos en los arreglos

Numpy tiene 5 tipos numéricos básicos, a saber:

- booleanos (`bool`),
- enteros (`int`) ,
- enteros sin signo (`uint`),
- punto flotante (`float`) y
- complejo (`complex`).

Los tipos con números en su nombre, por ejemplo `int64`, indican el tamaño en bits del tipo (es decir, cuántos bits se necesitan para representar un solo valor en la memoria)



Tipos de datos en los arreglos (II)

- Con el método `dtype` asociado a los arreglos de `numpy` podemos crear arreglos con el tipo de datos deseado o recuperar el tipo de datos que contiene el objeto.
- A continuación se pide el tipo de dato que contiene el arreglo `a` y se crea un objeto con datos tipo `float64`

```
a.dtype                                                    ①
ar1 = np.array([1, 2, 3], dtype=np.float64)                ②
print(ar1,ar1.dtype)
```

① Regresa el tipo de dato que contiene el arreglo `a`
② para crear un arreglo que contenga datos de tipo real `float` de 64 bits.

```
dtype('float64')
[1. 2. 3.] float64
```



Tipos de datos en los arreglos (III)

A continuación arreglos con otros tipos de datos

```
ar3 = np.array([1, 2, 3], dtype=np.int64)      ①
ar3 = np.array([1, 2, 3], dtype=np.int32)      ②
ar4 = np.array([1, 2, 3], dtype=np.int16)      ③
ar5 = np.array([1, 2, 3], dtype=np.int8)       ④
ar6 = np.array([1, 0, 1], dtype=np.bool_)      ⑤
print(ar6,ar6.dtype)
```

① Enteros de 64 bits.
② Enteros de 32 bits.
③ Enteros de 16 bits.
④ Enteros de 8 bits.
⑤ Booleanos, `True` o `False`.



Tipos de datos en los arreglos (IV)

Podemos hacer la conversión de un tipo a otro, por ejemplo, para convertir el arreglo `ar1`, que es de tipo `float`, a entero de 32 bits, procedemos de la siguiente forma

```
ar1int=ar1.astype(np.int32)
print(ar1int,ar1int.dtype)
```

```
[1 2 3] int32
```



Indexación



Indexación en arreglos de NumPy

- Los arreglos n—dimensionales de `numpy` se indexan usando la indexación estándar de Python (`x[obj]`), que hemos estudiado con tuplas y listas
- Hay diferentes tipos de indexación disponibles según el objeto: indexación básica, indexación avanzada y acceso de campo.



Indexación básica

- La *indexación de un solo elemento* funciona exactamente igual que para otras secuencias estándar de Python.
- Inicia en cero y acepta índices negativos para la indexación desde el final del arreglo.

```
a=np.arange(10)      ①
a[2]                 ②
a[-2]                ③
A[1,2]               ④
A[1,-3]              ⑤
```

② Tercer elemento del arreglo.
③ El segundo elemento, desde el final hacia adelante.
④ Ejemplo con arreglos bidimensionales: elemento en la segunda fila y tercera columna de la matriz A dada en 1, es decir, el elemento A_{23}.
⑤ Segunda fila, última columna de la matriz A.



Indexación básica (II)

Cuando se tiene un arreglo n—dimensional y se entrega un único valor, predeterminadamente el valor se toma como índice de la primera dimensión (en el caso de matrices, la fila), a continuación se selecciona la segunda fila de A.

```
A[1]          ①
A[1][2]       ②
```

① Segunda fila de A
② Tercer elemento de la segunda fila de A, equivalente a `A[1,2]`.

```
array([ 136, 2114, 4176, 3583])
4176
```



Segmentación

La *segmentación* se produce cuando en `x[obj]` `obj` es de la forma `inicio:fin:paso`, un entero o una tupla de objetos de segmentación

```
a=np.arange(10)          ①
a[1:7:2]                 ②
a[5:]                    ③
```

① Arreglo de 0 a 9.
② Selecciona el elemento con índice 1, luego suma dos (1+2), selecciona el elemento de índice 3, luego suma dos (3+2), así que selecciona el elemento de indice 5.
③ Del sexto elemento del arreglo a en adelante (índice 5 en adelante).

```
array([1, 3, 5])
array([5, 6, 7, 8, 9])
```



Herramientas de indexado dimensional

Las herramientas de indexado dimensional permiten seleccionar toda una dimensión. Por ejemplo, en el caso de matrices, toda una fila o conjunto de filas, o toda una columna o grupo de columnas. En general seleccionar submatrices, veamos algunos ejemplos con la matriz A dada en 1

```
A[1:,]          ①
A[:,1]          ②
A[(0,2),]       ③
```

① Selecciona de la fila dos (índice 1) de A en adelante.
② Selecciona la segunda columna de A.
③ Selecciona la fila 1 y 3 de A

```
array([[  136,  2114,  4176,  3583],
       [  269,  4176,  8257,  7104],
       [  260,  3583,  7104, 12276]])
array([ 137, 2114, 4176, 3583])
array([[   9,  137,  297,  261],
```

Indexación entera

- La indexación entera de arreglos permite la selección de elementos arbitrarios en el arreglo en función de su índice n—dimensional.
- Cada arreglo de enteros representa los índices en la correspondiente dimensión.
- Se permiten valores negativos en los arreglos de índices y funcionan como se explicó en la clase de listas.



Indexación entera (ejemplos)

```
a[np.array([3, 3, 0, 8])]                                    ①
a[np.array([3, 3, -3, 8])]                                   ②
A[np.array([0, 0, 3, 3]),np.array([0, 3, 0, 3])]             ③
A[np.ix_([0,1],[0,1])]                                       ④
A[0:2,0:2]                                                   ⑤
```

① Se seleccionan del arreglo a el elemento con índice 3 dos veces, luego el de índice cero y por ultimo el de índice 8.

② Se seleccionan del arreglo a el elemento con índice 3 dos veces, luego el tercer elemento de atrás hacia adelante y (7) por último el de índice 8.

③ Selecciona los elementos en las esquinas de la matriz A 1.

④ Selecciona la submatriz (menor) de 2×2 de la esquina superior izquierda de la matriz A.

⑤ Otra forma de conseguir el resultado anterior.

```
array([3, 3, 0, 8])
array([3, 3, 7, 8])
array([    9,   261,   260, 12276])
```



Indexación booleana

- Podemos usar como índice un arreglo de valores lógicos,
- este tipo de indexación es muy útil porque permite extraer del arreglo los valores que cumplan cierta condición.
- Por ejemplo,
  - extraer los números pares;
  - los que sean mayores que la media;
  - los que estén por debajo de primer cuartil;
  - entre otras condiciones, veamos como se hace



Indexación booleana (ejemplos)

```
a[a%2==0]   ①
a[a>5]      ②
a[a<=3]     ③
```

① Se extraen del arreglo a los elementos pares.
② Valores en el arreglo a mayores que 5.
③ Valores del arreglo a menores o iguales que 3

```
array([0, 2, 4, 6, 8])
array([6, 7, 8, 9])
array([0, 1, 2, 3])
```



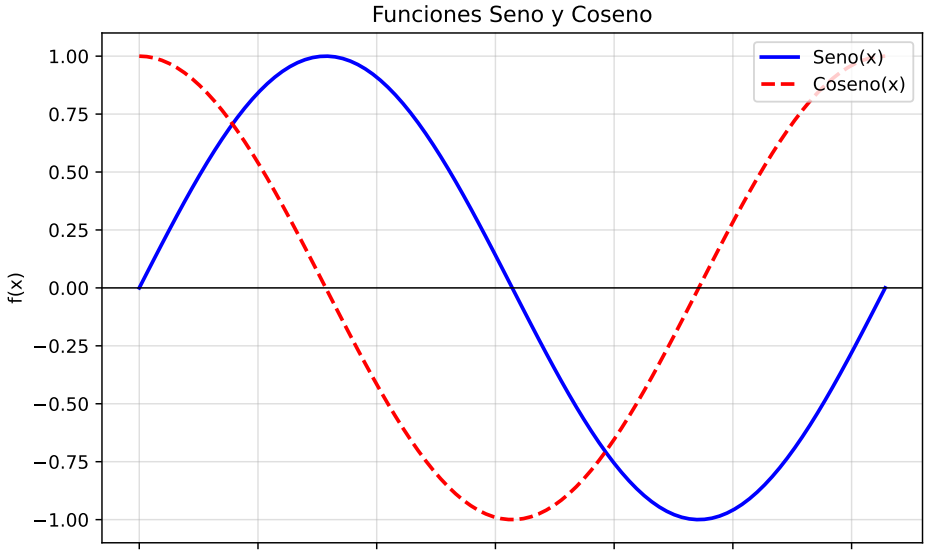
Residuo de la división

Nota

- En el código anterior se está usando el operador `%` que regresa el residuo de la división.
- Si al dividir un número por 2, el residuo es cero, entonces el número es par.
- De esta forma `a%2==0` regresa `True` para aquellos elementos del arreglo que sean pares y `False` en caso contrario.
- El resultado es que se extraen del arreglo `a` aquellos elementos que sean pares, es decir, los que se correspondan con `True` en el vector lógico.



Aplicación


Funciones Seno y Coseno
Seno(x)
Coseno(x)
f(x)
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
1
2
3
4
5
6

Aplicación

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Creación del dominio usando np.linspace
# Generamos 100 puntos equiespaciados entre 0 y 2*pi
x = np.linspace(0, 2 * np.pi, 100)

# 2. Evaluación de las funciones vectorizadas
# Al aplicar np.sin() a un arreglo, NumPy calcula el seno para cada elemen
y_seno = np.sin(x)
y_coseno = np.cos(x)

# 3. Inicialización y configuración del lienzo
plt.figure(figsize=(8, 5))

# Trazado de las curvas
plt.plot(x, y_seno, label="Seno(x)", color="blue", linewidth=2)
```

